WEBRUM-05: Error Analysis

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 5 of 10 | **Created:** March 2026 | **Last Updated:** 08/12/2026

Overview

JavaScript errors are among the most impactful issues affecting web application user experience. A single unhandled exception can break page functionality, prevent form submissions, or cause entire features to stop working. Dynatrace RUM captures JavaScript errors, XHR/fetch errors, and custom errors in real-time, enabling you to quantify error impact on real users.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	Web applications with error capture active
Permissions	`storage:events:read`
Previous Notebook	WEBRUM-01: RUM Fundamentals

1. Error Types in RUM

Dynatrace captures several categories of browser-side errors:

Error Type	Description	Example
JavaScript error	Unhandled exception or runtime error	`TypeError: Cannot read property 'x' of undefined`
XHR error	Failed XHR/fetch request (HTTP 4xx/5xx or network error)	`HTTP 500 on POST /api/checkout`
Custom error	Errors reported via the RUM API (`dtrum.reportError()`)	`Payment validation failed`
CSP violation	Content Security Policy blocking a resource	`Refused to load script from 'http://evil.com'`
Resource error	Failed resource loading (images, scripts, stylesheets)	`Failed to load /assets/app.js`

Error Data Model

Field	Description
`error.message`	Error message text
`error.type`	Error classification
`error.source`	Source file and line number

Field	Description
`action.name`	User action during which the error occurred
`sessionId`	Session containing the error
`application`	Application name

```
// Error / navigation vocabulary corrected 08/12/2026 (New RUM):
//   filter type == "Error"  -&gt; filter characteristics.has_error == true
(11,909 events; identical
//                              population to isNotNull(error.type), whose
values are request/csp/exception)
//   error.message           -&gt; error.reason
//   user_action.type == "RouteChange" -&gt; "same_view"  (the New RUM SPA
route-change value; the
//                              only other value is "hard_navigation".
"Custom" has NO equivalent.)
//   connection.type         -&gt; network.protocol.name
// CLASSIFIER MATTERS AS MUCH AS THE FIELD: navigation-timing fields
(performance.dom_interactive,
// performance.load_event_end) live on classifier "navigation" and are 0 on
"page_summary", so a
// page_summary filter silently empties them. ttfb.* is the opposite — it
lives on page_summary.
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; primary_tags.application
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.waiting_duration
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the "real_user" literal itself could NOT be confirmed here and is the
documented value form;
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Field vocabulary corrected 08/12/2026 — this series targets **New RUM**,
but was written
// against names that are null on New RUM data, so these cells returned
nothing while erroring
// nowhere. Verified against 5,556,127 user.events records (schema 0.24.0,
javascript agent):
//   action.type == "Load"             -&gt; characteristics.classifier ==
```

```
"navigation"
//   action.type                        -&gt; user_action.type
(hard_navigation | same_view)
//   action.name                        -&gt; page.detected_name
//   web_vitals.largest_contentful_paint-&gt; lcp.start_time        (327,099
populated)
//   web_vitals.cumulative_layout_shift -&gt; cls.value             (387,254
populated)
//   app.name                           -&gt; dt.rum.application.id
// UNITS CHANGE WITH THE FIELD. web_vitals.* was a nanosecond DURATION, so `/
1ms` was correct for
// it; lcp.start_time is a PLAIN NUMBER already in milliseconds, and dividing
it by 1ms yields
// null. Compare it against the 2500/4000 ms thresholds directly.
// `web_vitals.*` does still exist in the same schema, but carried 16 records
in 30 days against
// lcp.*'s 327,099 — it is not a different RUM generation, just a rarely-
populated sibling.
// Recent RUM errors — explore the data structure
fetch user.events, from:-1h
| filter characteristics.has_error == true
| fieldsKeep timestamp, error.reason, error.type, error.source,
page.detected_name, primary_tags.application, dt.rum.session.id
| sort timestamp desc
| limit 20
```

2. Error Volume and Trends

Start by understanding overall error volume and how it changes over time.

```
// Error count by type over last 24 hours
fetch user.events, from:-24h
| filter characteristics.has_error == true
| summarize error_count = count(), by:{error.type}
| sort error_count desc
```

```
// Error trend over 24 hours — hourly bucketed by error type
fetch user.events, from:-24h
| filter characteristics.has_error == true
| makeTimeseries error_count = count(), interval:1h, by:{error.type}
```

```
// Error volume by application — which apps have the most errors?
fetch user.events, from:-24h
| filter characteristics.has_error == true
| summarize error_count = count(),
    unique_errors = countDistinct(error.reason),
    affected_sessions = countDistinct(dt.rum.session.id),
    by:{primary_tags.application}
| sort error_count desc
```

3. Error Grouping and Top Errors

Error messages often contain variable data (stack traces, URLs, IDs). Grouping by error message helps identify the most frequent issues.

```
// Top 15 errors by frequency — the most common error messages
fetch user.events, from:-24h
| filter characteristics.has_error == true
| summarize error_count = count(),
    affected_sessions = countDistinct(dt.rum.session.id),
    by:{error.reason, error.type}
| sort error_count desc
| limit 15
```

```
// Errors by page — which pages generate the most errors?
fetch user.events, from:-24h
| filter characteristics.has_error == true
| filter isNotNull(page.detected_name)
| summarize error_count = count(),
    unique_errors = countDistinct(error.reason),
    by:{page.detected_name}
| sort error_count desc
| limit 10
```

4. Error Impact Analysis

Not all errors are equal. An error affecting 1 session is different from one affecting 10,000. Impact analysis quantifies the blast radius of each error.

```
// Error impact — errors ranked by number of affected sessions
fetch user.events, from:-24h
| filter characteristics.has_error == true
| summarize total_occurrences = count(),
    affected_sessions = countDistinct(dt.rum.session.id),
    by:{error.reason}
| sort affected_sessions desc
| limit 10
```

```
// Error rate per application — percentage of sessions with errors
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| summarize total_sessions = count(),
    error_sessions = countIf(error.count &gt; 0),
    by:{primary_tags.application}
| fieldsAdd error_rate_pct = round(toDouble(error_sessions) /
toDouble(total_sessions) * 100.0, decimals: 2)
| sort error_rate_pct desc
```

> **Tip:** Focus on errors with high session impact rather than high occurrence count. A single user hitting a retry loop can generate thousands of occurrences from one session.

5. XHR and Fetch Errors

XHR/fetch errors indicate backend API failures impacting the frontend. These are particularly critical in SPAs where the UI depends entirely on API responses.

```
// Unit trap (08/12/2026): New RUM timing fields such as lcp.start_time and
ttfb.waiting_duration
// are PLAIN NUMBERS already in milliseconds, not durations. `field / 1ms`
yields null on them —
// silently, so a chart of nulls looks like "no data". Compare and aggregate
them directly.
// XHR errors by action — identify failing API calls
fetch user.events, from:-24h
| filter characteristics.has_error == true
| filter error.type == "request"
| summarize error_count = count(),
    affected_sessions = countDistinct(dt.rum.session.id),
    by:{error.reason, page.detected_name}
| sort error_count desc
| limit 15
```

```
// XHR error trend — are backend errors increasing?
fetch user.events, from:-24h
| filter characteristics.has_error == true
| filter error.type == "request"
| makeTimeseries error_count = count(), interval:1h
```

6. Rage Click Detection

A **rage click** occurs when a user rapidly clicks the same element multiple times — a strong signal of frustration. This usually indicates:

- A button that appears clickable but is not responding
- A form submission that is silently failing
- A UI element that is loading too slowly
- A dead link or broken navigation element

Dynatrace captures rage clicks as user action properties. They can also be detected through rapid action sequences:

```
// Sessions with rage clicks — identify frustrated users
fetch user.events, from:-24h
| filter type == "RageClick"
| summarize rage_count = count(),
    by:{page.detected_name, primary_tags.application}
| sort rage_count desc
| limit 10
```

```
// Rage click trend over 7 days — is frustration increasing?
fetch user.events, from:-7d
| filter type == "RageClick"
| makeTimeseries rage_count = count(), interval:1d
```

7. Error-to-Session Correlation

Understanding how errors correlate with session outcomes (bounce, low engagement, failed conversion) helps prioritize fixes.

```
// Compare sessions with and without errors — engagement impact
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| fieldsAdd has_errors = if(error.count &gt; 0, "With Errors", else: "No
Errors")
| summarize session_count = count(),
    avg_actions = avg(user_action_count),
    avg_duration_sec = avg(duration / 1s),
    bounce_rate = countIf(user_action_count == 1),
    by:{has_errors}
| fieldsAdd bounce_pct = round(toDouble(bounce_rate) / toDouble(session_count)
* 100.0, decimals: 1)
```

```
// Errors by browser — are certain browsers more error-prone?
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| filter isNotNull(browser.name)
| summarize total = count(),
    with_errors = countIf(error.count &gt; 0),
    by:{browser.name}
| fieldsAdd error_rate = round(toDouble(with_errors) / toDouble(total) *
100.0, decimals: 1)
| filter total &gt; 10
| sort error_rate desc
| limit 10
```

8. Summary and Next Steps

In this notebook, we covered:

- **Error types** — JavaScript errors, XHR errors, custom errors, resource errors
- **Error volume and trends** — Tracking error rates over time
- **Error grouping** — Identifying the most frequent error messages
- **Impact analysis** — Measuring how many sessions each error affects
- **XHR/fetch errors** — Backend API failures visible from the frontend
- **Rage clicks** — Detecting user frustration through rapid repeated clicks
- **Error-session correlation** — How errors impact engagement and bounce rates

Next Steps

- **WEBRUM-06: Performance Analysis** — Page load waterfall and performance bottleneck identification
- **WEBRUM-07: Session Replay** — Visually investigate error-impacted sessions

References

- Configure error detection for web applications (DT docs)
- User actions in RUM Classic (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.